



4AI08 - STATISTICS: LINEAR MODELS

Practical Work 1

Leave-One-Out Cross-Validation

Full Name : Imane SIHAMI

Notebook and source code on [GitHub](#)

Contents

1	Introduction	1
2	Exercise 1: Deleted Residuals	1
2.1	Question 1: Decomposition after deleting one observation	1
2.2	Question 2: Relation involving the complete Gram matrix	2
2.3	Question 3: Sherman–Morrison formula	2
2.4	Question 4: Compact expression for the deleted estimator	3
2.5	Question 5: Deleted-residual identity	3
3	Exercise 2: Coding Leave-One-Out Cross-Validation	4
3.1	Part 1: OLS from Scratch	4
3.1.1	Question 1: Data, Gram matrix and invertibility	4
3.1.2	Question 2: OLS estimator	4
3.1.3	Question 3: Residual diagnostics and uncertainty	5
3.1.4	Question 4: Validation with scikit-learn	6
3.2	Part 2: Leave-One-Out Cross-Validation	7
3.2.1	Question 1: Efficient algorithm and computational cost	7
3.2.2	Question 2: Naive and efficient implementations	7
3.2.3	Question 3: Running-time comparison	9
4	Conclusion	10

1 Introduction

We consider the fixed-design Gaussian linear model

$$\mathbf{y} = X\boldsymbol{\theta} + \boldsymbol{\varepsilon}, \quad X \in \mathbb{R}^{n \times p}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I_n),$$

where X has full column rank. Its columns include the intercept whenever an intercept is used. The ordinary least-squares (OLS) estimator is

$$\hat{\boldsymbol{\theta}} = (X^\top X)^{-1} X^\top \mathbf{y}.$$

We denote by $x_i \in \mathbb{R}^p$ the feature vector associated with observation i ; therefore, $x_i^\top$ is the i -th row of X . This convention makes the dimensions explicit:

$$x_i^\top \in \mathbb{R}^{1 \times p}, \quad x_i \in \mathbb{R}^{p \times 1}.$$

The purpose of this practical work is twofold. First, we derive an analytical formula for the leave-one-out deleted residual. Second, we implement OLS and compare a naive LOOCV procedure with an efficient method that uses this formula and requires only one regression fit.

Notation used throughout the report

The hat matrix and the ordinary residuals are

$$H = X(X^\top X)^{-1} X^\top, \quad \hat{\boldsymbol{\varepsilon}} = \mathbf{y} - X\hat{\boldsymbol{\theta}}.$$

Its i -th diagonal entry is the leverage

$$h_{ii} = x_i^\top (X^\top X)^{-1} x_i.$$

2 Exercise 1: Deleted Residuals

After deleting observation i , let $X_{(i)}$ and $\mathbf{y}_{(i)}$ denote the reduced design matrix and response vector. The corresponding estimator and deleted residual are

$$\hat{\boldsymbol{\theta}}^{(i)} = (X_{(i)}^\top X_{(i)})^{-1} X_{(i)}^\top \mathbf{y}_{(i)}, \quad \hat{\varepsilon}_i^{(i)} = y_i - x_i^\top \hat{\boldsymbol{\theta}}^{(i)}.$$

The objective is to establish the shortcut

$$\boxed{\hat{\varepsilon}_i^{(i)} = \frac{\hat{\varepsilon}_i}{1 - h_{ii}}}.$$

2.1 Question 1: Decomposition after deleting one observation

Since the rows of X are $x_1^\top, \dots, x_n^\top$, the Gram matrix can be written as

$$X^\top X = \sum_{j=1}^n x_j x_j^\top.$$

Separating the contribution of observation i gives

$$X^\top X = \sum_{j \neq i} x_j x_j^\top + x_i x_i^\top = X_{(i)}^\top X_{(i)} + x_i x_i^\top.$$

Similarly,

$$X^\top \mathbf{y} = \sum_{j=1}^n x_j y_j = \sum_{j \neq i} x_j y_j + x_i y_i = X_{(i)}^\top \mathbf{y}_{(i)} + x_i y_i.$$

Therefore,

Answer to Question 1

$$X^\top X = X_{(i)}^\top X_{(i)} + x_i x_i^\top, \quad X^\top \mathbf{y} = X_{(i)}^\top \mathbf{y}_{(i)} + x_i y_i.$$

2.2 Question 2: Relation involving the complete Gram matrix

The second identity from Question 1 implies

$$X_{(i)}^\top \mathbf{y}_{(i)} = X^\top \mathbf{y} - x_i y_i.$$

Multiplying by $(X^\top X)^{-1}$ yields

$$\begin{aligned} (X^\top X)^{-1} X_{(i)}^\top \mathbf{y}_{(i)} &= (X^\top X)^{-1} X^\top \mathbf{y} - (X^\top X)^{-1} x_i y_i \\ &= \hat{\boldsymbol{\theta}} - (X^\top X)^{-1} x_i y_i. \end{aligned}$$

Hence,

Answer to Question 2

$$(X^\top X)^{-1} X_{(i)}^\top \mathbf{y}_{(i)} = \hat{\boldsymbol{\theta}} - (X^\top X)^{-1} x_i y_i.$$

2.3 Question 3: Sherman–Morrison formula

From Question 1,

$$X_{(i)}^\top X_{(i)} = X^\top X - x_i x_i^\top.$$

We use the Sherman–Morrison identity

$$(A - uv^\top)^{-1} = A^{-1} + \frac{A^{-1}uv^\top A^{-1}}{1 - v^\top A^{-1}u}.$$

With $A = X^\top X$ and $u = v = x_i$, we obtain

$$(X_{(i)}^\top X_{(i)})^{-1} = (X^\top X)^{-1} + \frac{(X^\top X)^{-1} x_i x_i^\top (X^\top X)^{-1}}{1 - x_i^\top (X^\top X)^{-1} x_i}.$$

Using $h_{ii} = x_i^\top (X^\top X)^{-1} x_i$ and multiplying by $X_{(i)}^\top \mathbf{y}_{(i)}$ gives

Answer to Question 3

$$\hat{\boldsymbol{\theta}}^{(i)} = \left[(X^\top X)^{-1} + \frac{(X^\top X)^{-1} x_i x_i^\top (X^\top X)^{-1}}{1 - h_{ii}} \right] X_{(i)}^\top \mathbf{y}_{(i)}.$$

2.4 Question 4: Compact expression for the deleted estimator

Let $A = (X^\top X)^{-1}$. From Question 3,

$$\hat{\boldsymbol{\theta}}^{(i)} = \left(A + \frac{A x_i x_i^\top A}{1 - h_{ii}} \right) X_{(i)}^\top \mathbf{y}_{(i)}.$$

Question 2 gives

$$A X_{(i)}^\top \mathbf{y}_{(i)} = \hat{\boldsymbol{\theta}} - A x_i y_i.$$

Substitution therefore leads to

$$\begin{aligned} \hat{\boldsymbol{\theta}}^{(i)} &= \hat{\boldsymbol{\theta}} - A x_i y_i + \frac{A x_i x_i^\top (\hat{\boldsymbol{\theta}} - A x_i y_i)}{1 - h_{ii}} \\ &= \hat{\boldsymbol{\theta}} - A x_i y_i + \frac{A x_i (x_i^\top \hat{\boldsymbol{\theta}} - h_{ii} y_i)}{1 - h_{ii}} \\ &= \hat{\boldsymbol{\theta}} + \frac{A x_i [-y_i (1 - h_{ii}) + x_i^\top \hat{\boldsymbol{\theta}} - h_{ii} y_i]}{1 - h_{ii}} \\ &= \hat{\boldsymbol{\theta}} - \frac{A x_i (y_i - x_i^\top \hat{\boldsymbol{\theta}})}{1 - h_{ii}}. \end{aligned}$$

Because $\hat{\varepsilon}_i = y_i - x_i^\top \hat{\boldsymbol{\theta}}$, we conclude that

Answer to Question 4

$$\hat{\boldsymbol{\theta}}^{(i)} = \hat{\boldsymbol{\theta}} - \frac{(X^\top X)^{-1} x_i \hat{\varepsilon}_i}{1 - h_{ii}}.$$

2.5 Question 5: Deleted-residual identity

By definition,

$$\hat{\varepsilon}_i^{(i)} = y_i - x_i^\top \hat{\boldsymbol{\theta}}^{(i)}.$$

Inserting the expression from Question 4 gives

$$\begin{aligned} \hat{\varepsilon}_i^{(i)} &= y_i - x_i^\top \hat{\boldsymbol{\theta}} + \frac{x_i^\top (X^\top X)^{-1} x_i}{1 - h_{ii}} \hat{\varepsilon}_i \\ &= \hat{\varepsilon}_i + \frac{h_{ii}}{1 - h_{ii}} \hat{\varepsilon}_i \\ &= \frac{\hat{\varepsilon}_i}{1 - h_{ii}}. \end{aligned}$$

Main theoretical result

Provided that the reduced OLS problem is well defined (equivalently, $1 - h_{ii} \neq 0$), the deleted residual is

$$\hat{\varepsilon}_i^{(i)} = \frac{\hat{\varepsilon}_i}{1 - h_{ii}}.$$

Thus all LOOCV residuals can be recovered from one OLS fit.

3 Exercise 2: Coding Leave-One-Out Cross-Validation

3.1 Part 1: OLS from Scratch

All computations in this part use elementary linear algebra from NumPy. The dataset is generated with the supplied function and a fixed random seed, which makes the reported numerical values reproducible.

3.1.1 Question 1: Data, Gram matrix and invertibility

The data contain $n = 50$ observations and 10 generated features. Because the generator adds an intercept column, X has $p = 11$ columns.

Listing 1: Generation of the data and analysis of the Gram matrix.

```
X, y, theta_star = gen_data(
    n_samples=50,
    n_features=10,
    return_coef=True,
    random_state=0
)

gram_matrix = X.T @ X
eigenvalues = np.linalg.eigvalsh(gram_matrix)

print("Shape of X:", X.shape)
print("Eigenvalues:", eigenvalues)
print("Smallest eigenvalue:", np.min(eigenvalues))
print("Rank of X:", np.linalg.matrix_rank(X))
print("Invertible:", np.all(eigenvalues > 0))
```

The eigenvalues obtained are

(11.9020, 29.2139, 32.1271, 39.1982, 40.9985, 42.5345,
51.2980, 64.2396, 66.9494, 80.1866, 89.8322).

Numerical conclusion

$$X \in \mathbb{R}^{50 \times 11}, \quad \text{rank}(X) = 11, \quad \lambda_{\min}(X^\top X) = 11.9020 > 0.$$

Every eigenvalue is positive, so $X^\top X$ is positive definite and therefore invertible.

3.1.2 Question 2: OLS estimator

The normal equations are

$$X^\top X \hat{\boldsymbol{\theta}} = X^\top \mathbf{y}.$$

Numerically, solving this linear system is preferable to explicitly computing $(X^\top X)^{-1}$.

Listing 2: OLS implementation based on the normal equations.

```
def ols_fit(X, y):
    """Return the OLS estimate theta_hat, with shape (p,)"""
    gram_matrix = X.T @ X
    right_hand_side = X.T @ y
    return np.linalg.solve(gram_matrix, right_hand_side)

theta_hat = ols_fit(X, y)
estimation_error = theta_hat - theta_star
print("Error norm:", np.linalg.norm(estimation_error))
```

Table 1 compares the true and estimated parameters. The estimates are close to the true coefficients, while exact equality is not expected because the response contains Gaussian noise.

Table 1: True parameters, OLS estimates and estimated standard errors.

Index	Interpretation	θ_j^*	$\hat{\theta}_j$	Standard error
0	Intercept	7.0000	6.1805	0.8864
1	Feature 1	40.3793	39.4008	0.8623
2	Feature 2	85.7125	85.9969	0.8064
3	Feature 3	27.1252	27.9865	0.8438
4	Feature 4	39.9812	39.7177	0.7607
5	Feature 5	67.1383	67.8228	0.8475
6	Feature 6	11.7316	10.6568	0.8348
7	Feature 7	97.9962	100.0275	0.9856
8	Feature 8	42.3706	43.4976	0.8336
9	Feature 9	45.7059	45.5795	0.8552
10	Feature 10	34.4718	34.2841	1.1690

The Euclidean norm of the estimation error is

$$\|\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}^*\|_2 = 3.0971.$$

3.1.3 Question 3: Residual diagnostics and uncertainty

The fitted values, residuals and unbiased noise-variance estimator are

$$\hat{\mathbf{y}} = X\hat{\boldsymbol{\theta}}, \quad \hat{\boldsymbol{\varepsilon}} = \mathbf{y} - \hat{\mathbf{y}}, \quad \hat{\sigma}^2 = \frac{\|\hat{\boldsymbol{\varepsilon}}\|_2^2}{n - p}.$$

The estimated covariance matrix and coefficient standard errors are

$$\widehat{\text{Cov}}(\hat{\boldsymbol{\theta}}) = \hat{\sigma}^2 (X^\top X)^{-1}, \quad \text{SE}(\hat{\theta}_j) = \sqrt{\hat{\sigma}^2 [(X^\top X)^{-1}]_{jj}}.$$

Listing 3: Residuals, variance estimate, standard errors and coefficient of determination.

```
n, p = X.shape
y_hat = X @ theta_hat
residuals = y - y_hat
```

```

RSS = residuals @ residuals
sigma2_hat = RSS / (n - p)

gram_matrix_inverse = np.linalg.inv(X.T @ X)
covariance_theta_hat = sigma2_hat * gram_matrix_inverse
standard_errors = np.sqrt(np.diag(covariance_theta_hat))

TSS = np.sum((y - np.mean(y)) ** 2)
R2 = 1 - RSS / TSS

orthogonality = X.T @ residuals
print("sigma^2:", sigma2_hat)
print("R^2:", R2)
print("||X^T residuals||:", np.linalg.norm(orthogonality))

```

Table 2: Principal OLS diagnostic quantities.

Quantity	Numerical value
Estimated variance $\hat{\sigma}^2$	29.1843
Estimated standard deviation $\hat{\sigma}$	5.4022
Coefficient of determination R^2	0.999342
$\ X^\top \hat{\varepsilon}\ _2$	2.79×10^{-12}

The data generator uses a noise standard deviation of 5, corresponding to a true variance of 25. The estimate $\hat{\sigma}^2 = 29.1843$ is reasonably close for a sample of only 50 observations. The very high R^2 indicates that the generated linear signal explains almost all observed variability.

Finally, the normal equations imply

$$X^\top X \hat{\theta} = X^\top y \implies X^\top (y - X \hat{\theta}) = X^\top \hat{\varepsilon} = \mathbf{0}.$$

The observed norm 2.79×10^{-12} is effectively zero up to floating-point rounding. Geometrically, the residual vector is orthogonal to the column space of X .

3.1.4 Question 4: Validation with scikit-learn

Because X already contains a column of ones, the option `fit_intercept=False` must be used.

Listing 4: Comparison with scikit-learn.

```

model = LinearRegression(fit_intercept=False)
model.fit(X, y)

theta_sklearn = model.coef_
R2_sklearn = model.score(X, y)

print(np.max(np.abs(theta_hat - theta_sklearn)))
print(np.isclose(R2, R2_sklearn))

```

Validation of the OLS implementation

The maximum absolute difference between the two coefficient vectors is

$$9.95 \times 10^{-14},$$

and both implementations return

$$R^2 = 0.9993416548.$$

The differences are at numerical precision, confirming that the NumPy implementation is correct.

3.2 Part 2: Leave-One-Out Cross-Validation

3.2.1 Question 1: Efficient algorithm and computational cost

The naive procedure deletes one observation, fits OLS on the remaining sample, and repeats this operation for every i . Its deleted residual is

$$\hat{\varepsilon}_i^{(i)} = y_i - x_i^\top \hat{\boldsymbol{\theta}}^{(i)}.$$

The analytical result of Exercise 1 allows the same residual to be computed as

$$\hat{\varepsilon}_i^{(i)} = \frac{\hat{\varepsilon}_i}{1 - h_{ii}}, \quad h_{ii} = x_i^\top (X^\top X)^{-1} x_i.$$

Efficient LOOCV algorithm

1. Fit OLS once on the complete dataset.
2. Compute the ordinary residuals $\hat{\varepsilon} = \mathbf{y} - X\hat{\boldsymbol{\theta}}$.
3. Compute all leverage values $h_{ii} = x_i^\top (X^\top X)^{-1} x_i$.
4. Return $\hat{\varepsilon}_i/(1 - h_{ii})$ for every observation.

For one OLS fit, forming $X^\top X$ costs $O(np^2)$ and solving the $p \times p$ system costs $O(p^3)$. The resulting complexities are summarised in Table 3.

Table 3: Approximate computational costs of the two LOOCV methods.

Method	Number of OLS fits	Approximate cost
Naive LOOCV	n	$O(n^2 p^2 + np^3)$
Efficient LOOCV	1	$O(np^2 + p^3)$

3.2.2 Question 2: Naive and efficient implementations

The formulas are stated before the implementations to make the link between theory and code explicit. For the naive method,

$$\hat{\boldsymbol{\theta}}^{(i)} = \left(X_{(i)}^\top X_{(i)} \right)^{-1} X_{(i)}^\top \mathbf{y}_{(i)}, \quad \hat{\varepsilon}_i^{(i)} = y_i - x_i^\top \hat{\boldsymbol{\theta}}^{(i)}.$$

For the efficient method,

$$\hat{\varepsilon}_i^{(i)} = \frac{\hat{\varepsilon}_i}{1 - h_{ii}}.$$

Listing 5: Naive and efficient LOOCV implementations.

```
def naive_loocv(X, y):
    """Return the n deleted residuals, refitting OLS n times."""
    n = X.shape[0]
    deleted_residuals = np.empty(n)

    for i in range(n):
        keep = np.ones(n, dtype=bool)
        keep[i] = False

        X_train = X[keep]
        y_train = y[keep]
        theta_i = ols_fit(X_train, y_train)

        y_prediction_i = X[i] @ theta_i
        deleted_residuals[i] = y[i] - y_prediction_i

    return deleted_residuals

def efficient_loocv(X, y):
    """Return the n deleted residuals from a single OLS fit."""
    theta_hat = ols_fit(X, y)
    residuals = y - X @ theta_hat

    gram_matrix_inverse = np.linalg.inv(X.T @ X)
    leverages = np.sum(
        (X @ gram_matrix_inverse) * X,
        axis=1
    )

    return residuals / (1 - leverages)
```

The vectorised leverage computation avoids constructing the full $n \times n$ hat matrix. Row i of X represents $x_i^\top$, so the code computes exactly $x_i^\top (X^\top X)^{-1} x_i$.

Listing 6: Numerical comparison of the two methods.

```
loo_naive = naive_loocv(X, y)
loo_efficient = efficient_loocv(X, y)

maximum_difference = np.max(
    np.abs(loo_naive - loo_efficient)
)
loocv_mse = np.mean(loo_efficient ** 2)

print("Maximum difference:", maximum_difference)
print("Methods agree:", np.allclose(loo_naive, loo_efficient))
print("LOOCV MSE:", loocv_mse)
```

LOOCV verification

The maximum absolute difference is

$$2.97 \times 10^{-13},$$

so the two methods agree up to floating-point precision. The mean squared leave-one-out error is

$$\text{MSE}_{\text{LOOCV}} = \frac{1}{n} \sum_{i=1}^n \left(\hat{\varepsilon}_i^{(i)} \right)^2 = 38.1226.$$

3.2.3 Question 3: Running-time comparison

The experiment uses 50 generated features, hence $p = 51$ after adding the intercept. For each sample size, both implementations are run ten times and the mean and standard deviation of the elapsed times are recorded.

Listing 7: Benchmark over increasing sample sizes.

```
ns = [100, 200, 400, 800, 1600]
n_features = 50
n_repeats = 10

naive_mean_times, efficient_mean_times = [], []
naive_std_times, efficient_std_times = [], []

for n in ns:
    X_n, y_n = gen_data(
        n_samples=n,
        n_features=n_features,
        random_state=0
    )

    naive_times = []
    efficient_times = []

    for _ in range(n_repeats):
        start = time.perf_counter()
        naive_loocv(X_n, y_n)
        naive_times.append(time.perf_counter() - start)

        start = time.perf_counter()
        efficient_loocv(X_n, y_n)
        efficient_times.append(time.perf_counter() - start)

    naive_mean_times.append(np.mean(naive_times))
    efficient_mean_times.append(np.mean(efficient_times))
    naive_std_times.append(np.std(naive_times))
    efficient_std_times.append(np.std(efficient_times))
```

Table 4: Mean running times over ten repetitions.

n	Naive mean (s)	Efficient mean (s)	Naive std. (s)	Efficient std. (s)
100	0.003852	0.000124	0.000282	0.000019
200	0.009612	0.000153	0.000115	0.000010
400	0.028347	0.000442	0.000411	0.000115
800	0.098126	0.000624	0.002328	0.000062
1600	0.359721	0.000962	0.006381	0.000071

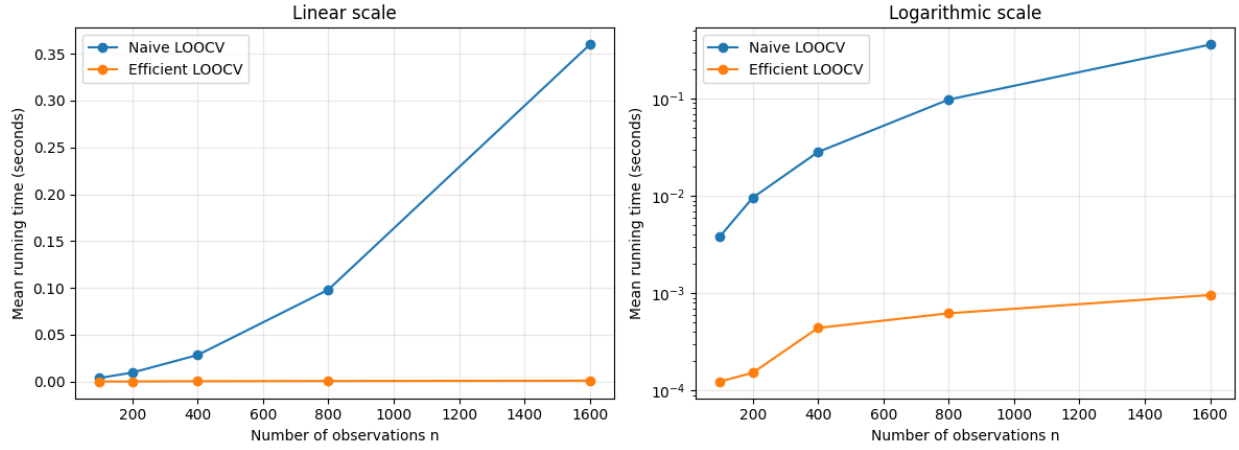


Figure 1: Mean running times of the naive and efficient LOOCV implementations. The left panel uses a linear scale and the right panel uses a logarithmic scale.

Figure 1 confirms the theoretical complexity comparison. The naive running time increases rapidly with n , whereas the efficient method remains below one millisecond in this experiment. At $n = 1600$, the naive method takes approximately 0.360 seconds and the efficient method approximately 0.000962 seconds, corresponding to a speed-up of about 374.

4 Conclusion

This practical work illustrates how linear-algebra identities can eliminate redundant computation. The OLS implementation based on the normal equations agrees with scikit-learn to numerical precision. The fitted residuals satisfy $X^\top \hat{\epsilon} \approx \mathbf{0}$, as required by the normal equations.

The main theoretical identity is

$$\hat{\epsilon}_i^{(i)} = \frac{\hat{\epsilon}_i}{1 - h_{ii}}.$$

It transforms LOOCV from n separate regressions into a single OLS fit followed by inexpensive leverage corrections. The naive and efficient residuals agree to approximately 3×10^{-13} , while the timing experiment shows an increasing computational advantage for the efficient method as the sample size grows.